



Лаборатории NVISO

Исследования в области кибербезопасности прямо из лаборатории! 🦊



≡ Меню

Перехват трафика Flutter на iOS и Android (HTTP/HTTPS/Dio Pinning)

Йерун Бекерс 📁 Android , iOS , Флаттер , Фрида ⌚ 18 августа 2022 г. ≡ 7 минут

Некоторое время назад я написал несколько статей о том, как использовать Flutter Man-In-The-Middle на [iOS](#) , [Android \(ARM\)](#) и [Android \(ARM64\)](#) . Эти посты были довольно популярны, и я часто возвращался к ним, чтобы скопировать эти скрипты самостоятельно.

Однако на прошлой неделе мы получили приложение Flutter, в котором скрипт больше не работал. Поскольку у нас был исходный код, было легко понять, что приложение использовало [пакет dio](#) для выполнения SSL Pinning.

Хотя можно было бы удалить логику закрепления и перекомпилировать приложение, было бы гораздо лучше, если бы мы могли просто отключить ее во время выполнения, чтобы нам не пришлось перекомпилировать себя.

TL;DR

- # Тестовое приложение

Приложение можно найти на странице GitHub, доступны сборки APK и IPA.

Логика закрепления Dio довольно проста:

```

1  ByteData data = await rootBundle.load('raw/certificate.
2  Dio dio = Dio();
3  (dio.httpClientAdapter as DefaultHttpClientAdapter).onH
4  SecurityContext sc = new SecurityContext();
5  sc.setTrustedCertificatesBytes(data.buffer.asUint8Lis
6  HttpClient httpClient = new HttpClient(context: sc);
7  return httpClient;
8  };
9
10 try {
11     Response response = await dio.get("https://www.nviso.
12     _status = "HTTPS: SUCCESS (" + response.headers.value
13 } catch (e) {
14     print("Request via DIO failed");
15     print("Exception: $e");
16     _status = "DIO: ERROR";
17 }

```

Новый подход

ssl_crypto_x509_session_verify_cert_chain функцию, которая в настоящее

время находится в [строке 361 ssl_x509.cc](#) . Этот метод отвечает за проверку цепочки сертификатов, поэтому, если этот метод возвращает true, цепочка сертификатов должна быть действительной и соединение принимается.

При выполнении MitM в тестовом приложении на Android ARM64 в logcat выводится следующая ошибка:

```
3540 3585 I flutter : Request via DIO failed
3540 3585 I flutter : Exception: DioError [DioErrorType.other]:
HandshakeException: Handshake error in client (OS Error:
3540 3585 I flutter :  CERTIFICATE_VERIFY_FAILED: self signed
certificate in certificate chain(handshake.cc:393))
3540 3585 I flutter : Source stack:
3540 3585 I flutter : #0    DioMixin.fetch
(package:dio/src/dio_mixin.dart:488)
3540 3585 I flutter : #1    DioMixin.request
(package:dio/src/dio_mixin.dart:483)
3540 3585 I flutter : #2    DioMixin.get
(package:dio/src/dio_mixin.dart:61)
3540 3585 I flutter : #3    _MyHomePageState.callPinnedHTTPS
(package:flutter_pinning_demo/main.dart:124)
3540 3585 I flutter : <asynchronous suspension>
3540 3585 I flutter : HandshakeException: Handshake error in client (OS
Error:
3540 3585 I flutter :  CERTIFICATE_VERIFY_FAILED: self signed
certificate in certificate chain(handshake.cc:393))
```

Flutter предоставляет нам полезную информацию: в цепочке сертификатов есть самоподписанный сертификат, который ему не нравится.

Исходный скрипт MitM перехватывает `session_verify_cert_chain`, и по какой-то причине перехваты так и не были запущены. Метод `session_verify_cert_chain` вызывается из [ssl_verify_peer_cert](#) на строке 386, а ошибка, показанная выше, возникает из `OPENSSL_PUT_ERROR` на строке 393:

```

366 uint8_t alert = SSL_AD_CERTIFICATE_UNKNOWN;
367 enum ssl_verify_result_t ret;
368 if (hs->config->custom_verify_callback != nullptr) {
369     ret = hs->config->custom_verify_callback(ssl, &ale
370     switch (ret) {
371         case ssl_verify_ok:
372             hs->new_session->verify_result = X509_V_OK;
373             break;
374         case ssl_verify_invalid:
375             // If !SSL_VERIFY_NONE!, the error is non-fata
376             if (hs->config->verify_mode == SSL_VERIFY_NONE
377                 ERR_clear_error();
378                 ret = ssl_verify_ok;
379             }
380             hs->new_session->verify_result = X509_V_ERR_AP
381             break;
382         case ssl_verify_retry:
383             break;
384     }
385 } else {
386     ret = ssl->ctx->x509_method->session_verify_cert_c
387         hs->new_session.get(), hs, &alert)
388         ? ssl_verify_ok
389         : ssl_verify_invalid;
390 }
391
392 if (ret == ssl_verify_invalid) {
393     OPENSSL_PUT_ERROR(SSL, SSL_R_CERTIFICATE_VERIFY_FA
394     ssl_send_alert(ssl, SSL3_AL_FATAL, alert);
395 }

```

The code path that is most likely taken, is that a `custom_verify_callback` is registered, which makes line 368 return true, and the callback executed on line 369 returns `ssl_verify_invalid`. The code then jumps to line 392 and the `ret` variable does equal `ssl_verify_invalid` so the alert is shown.

```

366 uint8_t alert = SSL_AD_CERTIFICATE_UNKNOWN;
367 enum ssl_verify_result_t ret;
368 if (hs->config->custom_verify_callback != nullptr) {
369     ret = hs->config->custom_verify_callback(ssl, &ale
370     switch (ret) {
371         case ssl_verify_ok:
372             hs->new_session->verify_result = X509_V_OK;
373             break;
374         case ssl_verify_invalid:
375             // If !SSL_VERIFY_NONE!, the error is non-fata
376             if (hs->config->verify_mode == SSL_VERIFY_NONE
377                 ERR_clear_error();
378                 ret = ssl_verify_ok;
379             }
380             hs->new_session->verify_result = X509_V_ERR_AP
381             break;

```

```

382         case ssl_verify_retry:
383             break;
384     }
385     } else {
386         ret = ssl->ctx->x509_method->session_verify_cert_c
387             hs->new_session.get(), hs, &alert)
388             ? ssl_verify_ok
389             : ssl_verify_invalid;
390     }
391
392     if (ret == ssl_verify_invalid) {
393         OPENSSL_PUT_ERROR(SSL, SSL_R_CERTIFICATE_VERIFY_FA
394         ssl_send_alert(ssl, SSL3_AL_FATAL, alert);
395     }

```

The easiest approach would be to hook the `ssl_verify_peer_cert` function and modify the return value to be `ssl_verify_ok`, which is 0. By hooking this earlier method, both the default SSL validation and any custom validation is disabled. Unfortunately, the `ssl_send_alert` function already triggers an error and so modifying the return value of `ssl_verify_peer_cert` would be too late.

Fortunately, we can just throw out the entire function and replace it with a `return 0` statement:

```

1  function hook_ssl_verify_peer_cert(address)
2  {
3      Interceptor.replace(address, new NativeCallback((pat
4          console.log("[+] Certificate validation disabled
5          return 0;
6      }, 'int', ['pointer', 'int']));
7  }

```

The only thing that's left is finding the actual location of the `ssl_verify_peer_cert` function.

Finding the offsets

Manually

The approach which was explained in the previous blogposts can be followed to identify the `ssl_verify_peer_cert` function:

- Find references to the string "x509.cc" and compare them to `x509.cc` to find `session_verify_cert_chain`

- Find references to the method you identified in order to identify `ssl_verify_peer_cert`

Both `x509.cc` and `handshake.cc` use the `OPENSSL_PUT_ERROR` macro which swaps in the file name and line number, which you can use to identify the correct functions.

By pattern matching

Alternatively, we can use Frida's pattern matching engine to search for functions that look very similar to the function from the demo app. The first bytes of a function are typically very stable, as long as the number of local variables and function arguments don't change. Still, different compilers may generate different assembly code (e.g. usage of different registers or optimisations) so we do need to have some wildcards in our pattern.

After downloading and creating multiple Flutter apps with different Flutter versions, I came to the following list:

```
iOS x64: FF 83 01 D1 FA 67 01 A9 F8 5F 02 A9 F6 57 03 A9 F4 4F 04 A9
FD 7B 05 A9 FD 43 01 91 F? 03 00 AA 1? 00 40 F9 ?8 1A 40 F9 15 ?5 4?
F9 B5 00 00 B4
```

```
Android x64: F? 0F 1C F8 F? 5? 01 A9 F? 5? 02 A9 F? ?? 03 A9 ?? ?? ?? ??
68 1A 40 F9
```

```
Android x86: 2D E9 FE 43 D0 F8 00 80 81 46 D8 F8 18 00 D0 F8 ?? 71
```

These patterns should only result in one hit in the `libFlutter` library and all match to the start of the `ssl_verify_peer_cert` function.

The final script

Putting all of this together gives the following script. It's one script that can be used on Android x86, Android x64 and iOS x64.

Check GitHub for the latest version

The script below may have been updated on the [GitHub repo](#).

```
1  var TLSValidationDisabled = false;
2  var secondRun = false;
3  if (Java.available) {
4      console.log("[+] Java environment detected");
5      Java.perform(hookSystemLoadLibrary);
6      disableTLSValidationAndroid();
7      setTimeout(disableTLSValidationAndroid, 1000);
8  } else if (ObjC.available) {
9      console.log("[+] iOS environment detected");
10     disableTLSValidationiOS();
11     setTimeout(disableTLSValidationiOS, 1000);
12 }
13
14 function hookSystemLoadLibrary() {
15     const System = Java.use('java.lang.System');
16     const Runtime = Java.use('java.lang.Runtime');
17     const SystemLoad_2 = System.loadLibrary.overload('
18     const VMStack = Java.use('dalvik.system.VMStack');
19
20     SystemLoad_2.implementation = function(library) {
21         try {
22             const loaded = Runtime.getRuntime().loadLi
23             if (library === 'flutter') {
24                 console.log("[+] libflutter.so loaded"
25                 disableTLSValidationAndroid();
26             }
27             return loaded;
28         } catch (ex) {
29             console.log(ex);
30         }
31     };
32 }
33
34 function disableTLSValidationiOS() {
35     if (TLSValidationDisabled) return;
36
37     var m = Process.findModuleByName("Flutter");
38
39     // If there is no loaded Flutter module, the setTi
40     if (m === null) {
41         if (secondRun) console.log("[!] Flutter module
42         secondRun = true;
43         return;
44     }
45
46     var patterns = {
47         "arm64": [
48             "FF 83 01 D1 FA 67 01 A9 F8 5F 02 A9 F6 57
49         ],
50     };
51     findAndPatch(m, patterns[Process.arch], 0);
```

```

52 }
53 }
54
55 function disableTLSValidationAndroid() {
56     if (TLSValidationDisabled) return;
57
58     var m = Process.findModuleByName("libflutter.so");
59
60     // The System.loadLibrary doesn't always trigger,
61     if (m === null) {
62         if (secondRun) console.log("[!] Flutter module
63         secondRun = true;
64         return;
65     }
66
67     var patterns = {
68         "arm64": [
69             "F? 0F 1C F8 F? 5? 01 A9 F? 5? 02 A9 F? ??
70         ],
71         "arm": [
72             "2D E9 FE 43 D0 F8 00 80 81 46 D8 F8 18 00
73         ]
74     };
75     findAndPatch(m, patterns[Process.arch], Process.ar
76 }
77
78 function findAndPatch(m, patterns, thumb) {
79     console.log("[+] Flutter library found");
80     var ranges = m.enumerateRanges('r-x');
81     ranges.forEach(range => {
82         patterns.forEach(pattern => {
83             Memory.scan(range.base, range.size, patter
84                 onMatch: function(address, size) {
85                     console.log('[+] ssl_verify_peer_c
86                     TLSValidationDisabled = true;
87                     hook_ssl_verify_peer_cert(address.
88                 }
89             });
90         });
91     });
92
93     if (!TLSValidationDisabled) {
94         if (secondRun)
95             console.log('[!] ssl_verify_peer_cert not
96         else
97             console.log('[!] ssl_verify_peer_cert not
98     }
99     secondRun = true;
100 }
101
102 function hook_ssl_verify_peer_cert(address) {
103     Interceptor.replace(address, new NativeCallback((p
104         return 0;
105     }, 'int', ['pointer', 'int']));
106 }

```


About the author



Jeroen Beckers

Jeroen Beckers is a mobile security expert working in the NVISO Software Security Assessment team. He is a SANS instructor and SANS lead author of the SEC575 course. Jeroen is also a co-author of OWASP Mobile Security Testing Guide (MSTG) and the OWASP Mobile Application Security Verification Standard (MASVS). He loves to both program and reverse engineer stuff.

[LinkedIn](#)

Tagged: android, flutter, ios, frida, pinning

Published by Jeroen Beckers

[View all posts by Jeroen Beckers](#)

< [Finding hooks with windbg](#)

[Cortex XSOAR Tips & Tricks – Creating indicator relationships in integrations](#) >

11 thoughts on “Intercept Flutter traffic on iOS and Android (HTTP/HTTPS/Dio Pinning)”

Pingback: [Intercept Flutter traffic on iOS and Android \(HTTP/HTTPS/Dio Pinning\) – Nviso Labs – Library 11: Antagonish Project Edition](#)

Pingback: [Intercepting Flutter traffic on iOS – Nviso Labs](#)

Pingback: [Intercepting Flutter traffic on Android \(ARMv8\) – Nviso Labs](#)

Pingback: [Intercepting traffic from Android Flutter applications – Nviso Labs](#)

Pingback: [We're celebrating our 10th anniversary! – Nviso Labs](#)

josé

June 9, 2023 at 8:42 am

apparently something changed in the last version(s) and this isn't working on new builds anymore.... either that or I'm doing something wrong. in any case, this is what I get for `ssl_verify_result()` (I assume, I hope I'm not wrong...) in an android app I'm pentesting:

```
FUN_006a3cb8 XREF[4]: 002f8550, 0038a71c(*),
```

```
FUN_00753530:0075387c(c),
```

```
FUN_00753530:007545fc(c)
```

```
006a3cb8 ff 43 01 d1 sub sp,sp,#0x50
```

```
006a3cbc fe 13 00 f9 str x30,[sp, #local_30]
```

```
006a3cc0 f6 57 03 a9 stp x22,x21,[sp, #local_20]
```

```
006a3cc4 f4 4f 04 a9 stp x20,x19,[sp, #local_10]
```

```
006a3cc8 e1 02 00 b4 cbz x1,LAB_006a3d24
```

but even then, the hook isn't working... yet.

Loading...

[↩ Reply](#)

Jeroen Beckers

June 9, 2023 at 10:13 am

Hi Jose! Can you open a ticket on the github repo?

<https://github.com/NVISOsecurity/disable-flutter-tls-verification>

Loading...

↩ Reply

josé

June 12, 2023 at 6:21 am

Nevermind, I did everything wrong... It's working for my app. I had some tool not work and messed up the analysis/RE.

Loading...

Pingback: [iOS](#) [Android](#) [Flutter](#) [HTTP/HTTPS/Dio Pinning](#) -

Pingback: [Unpacking Flutter hives – NVISO Labs](#)

Обратная ссылка: [Распаковка ульев Flutter - F1TYM1](#)

Оставить ответ



[Вакансии на домашней странице NVISO](#)

Информация и поддержка

info@nviso.eu

Вас взломали?

csirt@nviso.eu